

# Advance java annotations

1. **@Entity** – Marks a class as a database entity (mapped to a table).
2. **@Id** – Specifies the primary key of an entity.
3. **@Column** – Maps a field to a specific column in the table.
4. **@Data** – From Lombok; generates getters, setters, toString, equals, and hashCode automatically.
5. **@OneToOne** – Defines a one-to-one relationship between two entities.
6. **@OneToMany** – Defines a one-to-many relationship (one parent → many children).
7. **@ManyToOne** – Defines many-to-one relationship (many records → one parent).
8. **@ManyToMany** – Defines many-to-many relationship (uses a join table).
9. **@WebServlet** – Declares a servlet and maps it to a URL pattern.
10. **@Override** – Indicates a method overrides a superclass method.
11. **@Component** – Marks a class as a Spring-managed bean.
12. **@Autowired** – Automatically injects dependencies (DI).
13. **@Primary** – Specifies the preferred bean when multiple candidates exist.
14. **@Qualifier** – It is used to specify which exact bean to inject when multiple beans of the same type exist.
15. **@Configuration** – Marks a class as a configuration class for Spring beans.
16. **@ComponentScan** – Tells Spring where to scan for components.
17. **@Bean** – Declares a method that returns a Spring bean.
18. **@Lazy** – Initializes the bean only when it is first requested.
19. **@Scope** – Defines bean scope (singleton, prototype, etc.).
20. **@Controller** – Marks a class as a web controller (returns views).
21. **@RestController** – Combines @Controller + @ResponseBody (returns JSON).
22. **@RequestMapping** – Maps HTTP requests to handler methods.
23. **@ResponseBody** – Sends return data directly as HTTP response body.
24. **@ModelAttribute** – Binds request parameters to a model object.
25. **@RequestParam** – Extracts query/form parameters from request.
26. **@PathVariable** – Extracts values from URL path.
27. **@RequestBody** – Binds request body (JSON/XML) to an object.
28. **@SpringBootApplication** – Main Spring Boot annotation (combines config + scan + auto-config).
29. **@GetMapping** – Handles HTTP GET requests.
30. **@PostMapping** – Handles HTTP POST requests.
31. **@PutMapping** – Handles HTTP PUT requests.
32. **@PatchMapping** – Handles HTTP PATCH requests.
33. **@DeleteMapping** – Handles HTTP DELETE requests.
34. **@Service** – Marks a class as a service layer component.
35. **@Repository** – Marks a class as DAO and handles DB exceptions.
36. **@ExceptionHandler** – Handles specific exceptions in controller.
37. **@ControllerAdvice** – Global exception handling for all controllers (for views).
38. **@RestControllerAdvice** – Global exception handling for REST APIs (returns JSON).
39. **@NotNull** – Field must not be null.

40. **@NotEmpty** – Field must not be null or empty (collections/strings).
41. **@NotBlank** – String must not be null, empty, or only whitespace.
42. **@Valid** – Triggers validation on nested objects.
43. **@Size()** – Specifies size limits (min/max length or collection size).
44. **@Pattern()** – Validates string using regex pattern.
45. **@Email()** – Validates email format.
46. **@Min()** – Minimum numeric value allowed.
47. **@Max()** – Maximum numeric value allowed.
48. **@Positive()** – Value must be greater than 0.
49. **@PositiveOrZero()** – Value must be  $\geq 0$ .
50. **@Negative / @NegativeOrZero()** – Value must be  $< 0$  or  $\leq 0$ .
51. **@Past()** – Date must be in the past.
52. **@PastOrPresent()** – Date must be past or today.
53. **@Future()** – Date must be in the future.
54. **@FutureOrPresent()** – Date must be future or today.
55. **@AssertTrue** – Field must be true.
56. **@AssertFalse** – Field must be false.
57. **@Transient** - Is used to mark a field so that it is not persisted to the database.

## Questions on annotations

- What is the difference between **@Component**, **@Service**, **@Repository**, and **@Controller**?
- Difference between **@Autowired** and **constructor injection**? Which is better and why?
- Difference between **@Primary** and **@Qualifier**?
- What happens if **multiple beans of same type exist without @Primary/@Qualifier**?
- Can **@Autowired** work on **private fields**? How does Spring inject it internally?
- Difference between **@Bean** and **@Component**?
- When would you use **@Lazy** initialization?
- Explain different scopes in **@Scope** and real-time use cases.
- How does **@ComponentScan** work internally?
- What does **@SpringBootApplication** internally combine?
- Difference between **@Configuration** and **@Component**?
- Why are **@Bean** methods usually inside **@Configuration** class?
- What happens if you remove **@ComponentScan**?
- Can you have multiple **@Configuration** classes?
- Difference between **@Controller** and **@RestController**?
- Difference between **@RequestMapping** and **@GetMapping/@PostMapping**?
- When would you use **@PathVariable** vs **@RequestParam**?
- Difference between **@RequestBody** and **@ModelAttribute**?
- What happens if **@ResponseBody** is not used in REST API?
- Can we use **@RequestMapping** at **class level** and **method level** together?
- How does Spring map incoming request to correct method?
- Difference between **@PutMapping** and **@PatchMapping**?
- When should we use **@DeleteMapping** vs **soft delete logic**?
- What happens if two methods have same **@GetMapping** path?

- How do you handle **invalid request body** using **@RequestBody**?
- Difference between **@Entity** and **@Table**?
- Why is **@Id** **mandatory** in an entity?
- Difference between **@GeneratedValue** strategies (AUTO, IDENTITY, SEQUENCE)?
- Difference between **@OneToMany** and **@ManyToOne**?
- Which side owns the relationship in JPA? How do you define it?
- What is **mappedBy** in relationships?
- Difference between **@OneToOne** and **@ManyToOne** in real use case?
- What is the default fetch type in **@OneToMany** vs **@ManyToOne**?
- What problem does **@ManyToMany** create in real-world systems?
- Difference between **@ControllerAdvice** and **@RestControllerAdvice**?
- How does **@ExceptionHandler** work internally?
- Can we define multiple **@ExceptionHandler** methods?
- How do you create **global exception handling** in **Spring Boot**?
- What happens if exception is not handled anywhere?
- Difference between **@NotNull**, **@NotEmpty**, and **@NotBlank**?
- When should we use **@Valid** annotation?
- What happens if **@Valid** is **missing** in controller?
- Difference between **@Size** and **@Pattern**?
- How does Spring return validation errors?
- How to customize validation error messages?
- Difference between **@Min/@Max** and **@Positive/@Negative**?
- When would you use **@Past** vs **@PastOrPresent**?
- Difference between **@WebServlet** and **Spring @Controller**?
- How does Spring internally replace traditional servlets?
- If your **bean is not getting injected**, what could be the reasons?
- How do you resolve **circular dependency** with **@Autowired**?
- What happens when **two APIs have same URL mapping**?
- How do you handle **versioning** in **@RequestMapping**?
- If a field fails validation, how do you **send proper error response to client**?
- How do you design a **scalable entity relationship** using **@OneToMany/@ManyToOne**?
- When would you avoid using **@ManyToMany**?
- How do you handle **lazy loading issues** (**LazyInitializationException**)?
- What are advantages and disadvantages of **@Data**?
- Why is **@Data** **sometimes avoided** in production?